File Management

Lecture 2

From Logical to Physical

The Disk as a Resource:

- A disk is divided into blocks (typically 512 bytes to 4KB).
- The file system must manage these blocks efficiently:
  a. allocate space to files
  b. keep track of which blocks belong to which file, and know which blocks are free.

File Allocation Methods

2.1 Contiguous Allocation

- Each file occupies a set of contiguous blocks on the disk.
- The directory entry for a file stores the starting block and the length (in blocks).

Example:

- File `A` starts at block 2, length 3 blocks → occupies blocks 2, 3, 4.
- File `B` starts at block 5, length 2 blocks → occupies blocks 5, 6.

2.1 Contiguous Allocation

Advantages:

- Simple implementation: Only starting block and length needed.
- Fast sequential access: To read the next block, just increment the current block number.
- Fast direct access: Given block `i` of the file, we can compute physical block = start + i.

Disadvantages:

- External fragmentation: As files are created and deleted, free space is broken into small chunks. May need compaction (expensive).
- Difficult to grow files: If a file needs more space and the next block is taken, the file cannot grow easily. One solution is to preallocate the maximum possible size (wastes space) or move the file (costly).

Use Cases: Contiguous allocation is used in some simple systems, optical media (CD/DVD). Not common in general-purpose OS today.

2.2 Linked Allocation

Concept:

- Each file is a linked list of disk blocks. Blocks may be scattered anywhere on the disk.
- Each block contains a pointer to the next block of the file.
- The directory entry contains a pointer to the first and last block (or just the first).

Example:

- File `A` starts at block 4 → block 4 points to block 7 → block 7 points to block 2 → end (null).

Advantages:

- No external fragmentation: Any free block can be used.
- Easy file growth: Just add a new block at the end by updating the last block's pointer.

2.2 Linked Allocation

Disadvantages:

- Only sequential access: To access block `i`, you must follow the pointers from the start. No efficient direct access.
- Reliability: If a pointer is lost or damaged, the rest of the file is lost.
- Space overhead for pointers: Each block needs a few bytes for the pointer (reduces effective data space). However, this is usually small relative to block size.

Variation: File Allocation Table (FAT)

- Used in MS-DOS and early Windows.
- Instead of storing pointers in the blocks themselves, a separate table (the FAT) holds the pointer information for the entire disk.
- Each disk block has an entry in the FAT that indicates the next block of the file, or a special end-of-file marker.
- Advantages: Faster random access (by consulting the FAT in memory) and improved reliability (the FAT can be replicated). However, the FAT itself can be large.

2.3 Indexed Allocation

Concept

- All pointers to a file's blocks are collected into a single location: the index block.
- The directory entry points to the index block.
- The index block is an array of disk addresses; the i-th entry points to the i-th block of the file.

Example:

- File `A` has index block at block 8. Index block contains: [19, 3, 15, 0, ...] meaning file blocks 0→19, 1→3, 2→15, etc.

Advantages:

- Supports direct access: To access block i, just read the i-th entry from the index block and then that data block.
- No external fragmentation (blocks can be scattered).
- Easy file growth: If the index block becomes full, you can allocate another index block (linked or multi-level indexing).
- If the file is larger, it uses a single indirect pointer (points to a block of pointers), then double indirect, etc. This allows very large files while keeping the inode compact for small files.

2.3 Indexed Allocation

Disadvantages:

- Overhead: The index block itself consumes disk space. For a tiny file (e.g., 1 block), you still need an entire index block (wasteful).
- Performance: For large files, a single index block may not hold enough pointers. Solutions:
  - Linked scheme: Link multiple index blocks.
  - Multi-level indexing: Use a master index block pointing to secondary index blocks (like Unix inodes).
  - Combined scheme: Some systems (e.g., Unix) use a mix: a few direct pointers, plus single, double, and triple indirect pointers.

Unix inode example:

- The inode contains a small number of direct block pointers.
- If the file is larger, it uses a single indirect pointer (points to a block of pointers), then double indirect, etc. This allows very large files while keeping the inode compact for small files.

Refe: https://www.expertsmind.com/questions/index-node-%28inode%29-3018349.aspx

Free Space Management

3.1 Bitmap / Bit Vector

Concept:

- A sequence of bits, one per disk block.
- 1 = free, 0 = allocated (or vice versa).
- The bitmap is stored on disk (and cached in memory for efficiency).

Example:

- Disk with 16 blocks: bitmap `1100110011110000` means blocks 0,1 are free? Actually depends on convention.

Free Space Management

3.1 Bitmap / Bit Vector

Advantages:

- Simple, space-efficient for small to medium disks.
- Easy to find contiguous free blocks (scan for runs of 1's).

Disadvantages:

- Size: For a large disk, the bitmap itself can be large (e.g., 1TB disk with 4KB blocks → 32 million bits → 4MB). Not huge but must be managed.
- Must keep in memory for performance, but can be paged.

Free Space Management

3.2 Linked List (Free List)

Concept:

- Link together all free blocks, with each free block containing a pointer to the next free block.
- A special pointer (head) points to the first free block.

Advantages:

- Only one pointer needed in memory (the head).
- No extra space dedicated to a bitmap.

Disadvantages:

- Traversing the list is slow (requires reading many blocks). Not efficient for finding contiguous space.
- Only good for allocating one block at a time.

Free Space Management

3.3 Grouping

Concept:

- Store pointers to many free blocks in a single block (like indexing).
- For example, the first free block contains addresses of the next n free blocks; then among those, the last one points to another block of pointers, etc.
- This is essentially a linked list of groups of free block addresses.

Advantages:

- Faster traversal than a simple linked list (can read many pointers at once).
- Good for finding contiguous runs.

References

4.3.2